

# Shared-Memory SPSC IPC: A Controlled Study of Wakeup Mechanisms and io\_uring Modes

Rahul Raman

Department of Computer Science and  
Engineering  
International Institute of Information  
Technology Bangalore  
Bangalore, India

Yuvraj Deshmukh

Department of Computer Science and  
Engineering  
International Institute of Information  
Technology Bangalore  
Bangalore, India

B. Thangaraju

Department of Computer Science and  
Engineering  
International Institute of Information  
Technology Bangalore  
Bangalore, India

**Abstract**—Inter-Process Communication (IPC) performance is a foundational bottleneck in concurrent, multi-process software systems. Traditional kernel-mediated mechanisms—POSIX Pipes, UNIX Domain Sockets, and POSIX Message Queues—require system calls and kernel-mediated data movement and may incur scheduler transitions; in our depth-1 suite, their 64 B median single-trip latencies are 5.29–5.57  $\mu$ s. Modern high-performance systems reduce this cost by employing lock-free Single-Producer Single-Consumer (SPSC) shared-memory ring buffers that transfer payload data entirely in userspace via memcopy into POSIX shared memory regions. A critical and largely unanswered question, however, is: when the shared ring empties and the consumer must sleep, which wakeup primitive should be used—and what does Linux’s io\_uring framework specifically contribute over simpler alternatives such as futex or eventfd?

This paper presents a rigorous empirical measurement study that decouples payload transport from wakeup-mechanism signaling and applies a two-suite evaluation framework: (1) a wakeup-mechanism ablation comparing six coordination strategies (busy\_poll, spin\_backoff, adaptive, futex, eventfd, io\_uring) on an identical cache-aligned SPSC ring under saturated, bursty, and offered-load traffic conditions across a 64 B–1 MiB message-size sweep; and (2) a corrected depth-1 ping-pong latency suite in which both the send and receive timestamps are recorded on the single initiator core using CLOCK\_MONOTONIC\_RAW, eliminating cross-core TSC drift and in-flight pipelining artefacts, reporting full percentile distributions (P50, P90, P99, P99.9) using size-scaled samples of 2 000–100 000 round-trips per supported configuration.

Our results separate the roles of the data and wakeup paths. At 64 B, adaptive and busy-poll waiting achieve median single-trip latencies of 0.345 and 0.381  $\mu$ s, respectively; the pipe and socket medians are 13.9–15.9 $\times$  higher. In the controlled saturated ablation, the six primary variants span 13.37–15.75 GiB/s at 64 KiB, with interrupt-mode io\_uring at 13.37 GiB/s. In depth-1 ping-pong tests (Suite 2), io\_uring incurs a modest per-event ring submission overhead (+3.15  $\mu$ s over futex at 64 B). However, under bursty wakeup conditions (Suite 1), where wakeups occur when the ring empties, io\_uring performs on par with futex and eventfd (1.092–1.095 ms median wakeup latency). Thus, the shared-memory data path dominates saturated throughput, whereas the wait strategy dominates empty-ring latency and CPU cost. A frequency-pinned replication (performance governor and no\_turbo=1) reproduces the close ordering of the kernel-sleeping variants. We additionally evaluate SQPOLL as a separate configuration: its 64 B bursty wakeup median (1058.5  $\mu$ s) is close to interrupt-mode io\_uring (1095.0  $\mu$ s), rather than a decisive improvement.

**Index Terms**—io\_uring, IPC, shared memory, SPSC ring buffer, lock-free, wakeup mechanism, latency, throughput, Linux kernel, systems performance

## I. INTRODUCTION

Modern software infrastructure—spanning high-frequency trading (HFT) matching engines, real-time database kernels, media processing pipelines, and containerised microservice mesh proxies—relies heavily on efficient Inter-Process Communication. Even a modest per-message overhead of a few microseconds accumulates to tens of milliseconds when millions of messages flow per second. At 100 kmsg/s, a 3  $\mu$ s per-message overhead consumes approximately 30 % of one CPU core; at 10 Mmsg/s it would require about 30 core-seconds of work per second.

Linux provides a rich set of mature IPC primitives. POSIX FIFOs and UNIX domain sockets route all data through the kernel, imposing context switches and double memory copies on every message. POSIX message queues add priority ordering but retain kernel-mediated delivery semantics. Shared memory avoids the data-copy penalty by mapping identical physical pages into both producer and consumer address spaces; the kernel is contacted only for initial setup.

The cost of these kernel interactions is non-trivial and grows with message rate. A write/read pair introduces fixed entry, copy, and scheduling costs before payload-dependent work is considered. At high message rates, these per-message costs can consume several cores and become the dominant budget for latency-sensitive applications.

Lock-free SPSC ring buffers—popularised by the LMAX Disruptor [5] and Intel DPDK [6]—take shared memory a step further by coordinating access using only atomic load/store instructions on cache-line-aligned indices, avoiding any kernel involvement on the data path. Message delivery in the uncontended case reduces to a memcopy plus two atomic index updates; the measured adaptive P50 is 345 ns at 64 B, more than an order of magnitude below the blocking baselines in this experiment.

A remaining challenge is the *empty-ring wakeup problem*: when the ring is empty, busy-spinning wastes a full CPU core; sleeping the consumer requires a kernel-level wakeup mechanism.

Linux’s io\_uring interface (introduced in kernel 5.1 [1]) has attracted substantial interest for its promise of zero-syscall I/O. A growing body of practitioner literature presents “io\_uring-based IPC rings” as offering superior latency

and throughput over both traditional IPC and simpler shared-memory approaches. However, these claims conflate two independent design decisions: (1) the payload data path choice (shared memory vs. kernel-copy), and (2) the wakeup mechanism choice (`io_uring` vs. `futex` vs. `spin`). No prior measurement study isolates these dimensions on identical hardware with a controlled ablation.

### Contributions

- 1) **Controlled wakeup ablation.** Six pluggable wakeup strategies are implemented on a bit-identical SPSC ring buffer; their impact on latency, throughput, and CPU time per message are measured across a 64 B–1 MiB payload sweep.
- 2) **Corrected depth-1 ping-pong protocol.** Queue depth is enforced at 1 and both round-trip endpoints are timestamped on the same core with `CLOCK_MONOTONIC_RAW`, eliminating TSC skew and pipelining artefacts.
- 3) **Tail-latency characterisation.** P50, P90, P99, P99.9, and 95 % confidence intervals are reported for every supported payload size, including 64 KiB (10 000 rounds) and 1 MiB (2 000 rounds). The 64 B and 4 KiB configurations use 100 000 and 50 000 rounds, respectively.
- 4) **Explicit measurement scope.** We report interrupt-mode and `SQPOLLRING` `io_uring` as distinct configurations, and record fixed-frequency settings for the low-load wakeup measurements.
- 5) **Reproducible artifact.** All source code, raw CSV data, and figure-generation scripts are published at [https://github.com/Rahul09123/io\\_uring-IPC](https://github.com/Rahul09123/io_uring-IPC).

## II. BACKGROUND & RELATED WORK

### A. Traditional Kernel-Mediated IPC

**POSIX Pipes (FIFOs)** (Stevens and Rago [2]) provide uni-directional byte streams backed by a kernel circular buffer (default 64 KiB, tunable to 1 MiB via `fcntl(F_SETPIPE_SZ)`). Every `write` copies data from user to kernel space; every `read` copies it back. The producer and consumer alternate between running and sleeping states as the pipe buffer fills and drains, incurring scheduler-wakeup latency on each transition. At very small messages (64 B), the pipe’s fixed per-syscall entry cost dominates; at large messages ( $\geq 256$  KiB), finite pipe capacity creates producer backpressure.

**UNIX Domain Sockets** bypass the network stack but retain the socket buffer abstraction (`sk_buff`). Each `sendmsg/recvmmsg` pair requires a system call and a socket-layer serialisation lock (`mutex_lock` visible in flamegraph traces). Socket send/receive buffers are tuned to 2 MiB via `SO_SNDBUF/SO_RCVBUF` in our implementation. Despite the per-call overhead, UNIX sockets achieve competitive throughput at large payload sizes in the recorded runs. The experiment does not isolate which socket-buffer or flow-control behavior produces that result.

**POSIX Message Queues (`mqueue`)** deliver structured, priority-sorted messages via a virtual filesystem inode under

TABLE I  
KERNEL-IPC SYSTEM-CALL AND COPY COSTS PER MESSAGE

| Mechanism   | Syscalls<br>/msg | Copies<br>/msg | Kernel buffer                         |
|-------------|------------------|----------------|---------------------------------------|
| POSIX Pipe  | 2                | 2              | Ring buf<br>(64 KiB–1 MiB)            |
| UNIX Socket | 2                | 2              | <code>sk_buff</code> (up to<br>2 MiB) |
| POSIX MQ    | 2                | 1–2            | VFS inode (10<br>msgs)                |
| SHM Ring    | 0                | 1              | None (userspace)                      |

`/dev/mqueue`. A key optimisation—`pipelined_send`—delivers directly into a blocked receiver’s address space when one is already waiting, bypassing intermediate queue staging and making `mqueue` competitive with shared-memory rings at small message sizes. However, the default maximum message size is 8 KiB and the maximum queue depth is 10 messages, both controlled by kernel tunables (`fs.mqueue.msgsize_max`, `fs.mqueue.msg_max`), imposing hard architectural limits at scale.

### B. Lock-Free SPSC Shared-Memory Rings

Shared-memory IPC maps identical physical pages into producer and consumer address spaces via `shm_open` and `mmap`. SPSC ring buffers (Lamport [4], Herlihy and Shavit [9], LMAX Disruptor [5], DPDK `rte_ring` [6]) coordinate using a pair of atomic integer indices. The critical correctness rule is: the producer advances `head` with `release` ordering after writing payload; the consumer reads `head` with `acquire` ordering before consuming.

The C++17 memory model formally defines these orderings. A *release store* ensures all prior writes are visible to any thread that subsequently performs an *acquire load* of the same variable. For an SPSC ring, this means the consumer observing `head > tail` is guaranteed to see the complete payload written by the producer before `head` was advanced. Using `memory_order_relaxed` on `head` (a common optimisation) works on x86 due to Total Store Order (TSO) but is technically undefined behaviour on weakly-ordered architectures (ARM, RISC-V, Power) that permit Store-Load reordering. We use `memory_order_seq_cst` on the publish store to maintain portability and prevent the lost-wakeup race.

**False sharing**—where producer and consumer cache lines collide—is prevented by placing `head`, `tail`, and control flags on separate 64-byte-aligned cache lines using `alignas(64)`. Without this separation, a producer store to `head` invalidates the cache line containing `tail` on the consumer core, forcing a cache-coherency RFO (Request For Ownership) on every slot exchange and doubling effective memory latency.

### C. The `io_uring` Interface

Introduced by Jens Axboe [1] and merged in Linux 5.1, `io_uring` exposes two ring buffers shared between userspace and the kernel: the Submission Queue (SQ) and the Completion Queue (CQ). In the default interrupt-driven mode (`flags=0`),

a single `io_uring_enter` system call submits all pending SQEs and optionally waits for completions. In `SQPoll` mode (`IORING_SETUP_SQPOLL`), a dedicated kernel poller thread can eliminate submission syscalls; completion waiting and host policy still determine whether other kernel entries occur.

In our implementation, `io_uring` is used *exclusively for the wakeup signaling path*. When the consumer sleeps, it submits an `IORING_OP_READ` on a named FIFO and blocks in `io_uring_submit_and_wait`. When the producer detects `consumer_sleeping = 1`, it submits an `IORING_OP_WRITE` to the same FIFO. The payload data path is pure `memcpy` into shared memory—`io_uring` never touches message bytes.

#### D. Prior Work on `io_uring` Performance

Didona et al. [7] compared `libaio`, `SPDK`, and `io_uring` for storage I/O, demonstrating that polling makes `io_uring` competitive with `SPDK` for sequential workloads. Ren and Trivedi [8] demonstrated that `io_uring` can match kernel-bypass performance for certain NVMe access patterns. Neither study addresses the question of wakeup-primitive selection for shared-memory IPC, leaving this dimension unexplored. Our work fills this gap with a controlled ablation study on a single fixed ring design.

### III. SYSTEM DESIGN

Our architecture cleanly separates the data path from the control path.

#### A. Lock-Free SPSC Ring Buffer Layout

The shared ring buffer is a single POSIX shared memory object opened via `shm_open` and mapped with `mmap` into both the producer and consumer address spaces.

Listing 1. Cache-aligned SPSC RingBuffer (common.h)

```
struct alignas(64) RingBuffer {
    alignas(64) atomic<uint64_t> head;    // Producer write
    index
    alignas(64) atomic<uint64_t> tail;    // Consumer read
    index
    alignas(64) atomic<uint32_t> consumer_sleeping;

    struct alignas(64) Slot {
        uint64_t send_ns;                // Producer timestamp
        (Suite 1 clock)
        uint64_t wakeup_publish_ns;      // Wakeup-signal timestamp
        uint32_t size;                   // Payload bytes written
        char data[MAX_PAYLOAD];          // Up to 1 MiB of payload
    };
    Slot slots[NUM_SLOTS];               // Fixed 64-slot ring
    (~64 MiB total)
};
```

Three design properties are critical.

**(1) Cache-line isolation.** `head`, `tail`, and `consumer_sleeping` each occupy their own 64-byte cache line. Without this separation, a producer write to `head` invalidates the cache line containing `tail` on the consumer core, forcing an RFO (Request For Ownership) round-trip on every slot transition. Our hardware counter data (Table IX) are consistent with lower data-path cache pressure: the SHM Ring’s 4.97 % LLC

miss rate compares favourably to 30–34 % for the measured pipe and socket baselines.

**(2) Sequential consistency at publish boundaries.** `head` is stored with `memory_order_seq_cst` to prevent Store-Load reordering that could cause the producer to read stale `consumer_sleeping = 0` and skip sending a wakeup, silently deadlocking the consumer. On x86 (TSO), this is equivalent to a plain store followed by a `mfence`; on ARM64 it compiles to a `stlr` (store-release) instruction.

**(3) Lost-wakeup prevention.** A classic race exists in any producer-consumer pair: the consumer checks `head == tail` (empty), decides to sleep, but before setting `consumer_sleeping`, the producer writes a slot and checks `consumer_sleeping = 0` (decides not to signal), then the consumer sets `consumer_sleeping = 1` and waits—forever. We eliminate this race by having the consumer set `consumer_sleeping = 1` *first*, then re-check `head != tail` under a full memory fence. If a slot arrived in the window, the re-check catches it and the consumer proceeds without sleeping.

**Data-flow walkthrough.** On every message transfer: (a) the producer calls `memcpy` into `slots[head % NUM_SLOTS].data`, writes the timestamp and size, then executes a `seq_cst` store to `head`; (b) the consumer spin-reads `head` with acquire ordering; once it observes `head > tail`, it processes the slot and advances `tail` with a release store; (c) if the consumer finds the ring empty after its deadlock-prevention recheck, it invokes the configured wakeup primitive to sleep. Steps (a) and (b) involve zero system calls; step (c) invokes at most one system call per wakeup event.

#### B. Pluggable Wakeup Layer: Six Ablation Variants

To isolate the contribution of the wakeup mechanism, we implement six interchangeable wakeup modules on a single, bit-identical SPSC ring buffer data structure. Each primitive is invoked via its standard Linux kernel system interface (Linux man-pages [10]): Only the functions `consumer_wait()` and `producer_signal()` differ, guaranteeing that any observed latency difference is attributable solely to the wakeup mechanism.

*a) 1. busy\_poll.* The consumer spins in a tight while(`head.load(relaxed) == tail`) loop with no backoff. Zero system calls are issued while waiting. The measured 64 B depth-1 P50 is 0.381  $\mu$ s, at the cost of continuously occupying the pinned consumer core.

*b) 2. spin\_backoff.* Same tight spin with an `_mm_pause()` intrinsic on every iteration. `_mm_pause` inserts a brief pipeline hint on x86 to make sustained spinning friendlier to the execution pipeline; this study does not measure package power directly.

*c) 3. adaptive.* The consumer spins for up to 500 iterations; if no data arrives, it falls back to a `futex` sleep. This hybrid targets low-latency workloads where the ring is rarely empty (spin wins) while gracefully handling burst-idle traffic (`futex` sleep saves power).

TABLE II  
WAKEUP MECHANISM PROPERTIES SUMMARY

| Variant    | Wait Strategy   | Idle Behavior   | Kernel Wait Path |
|------------|-----------------|-----------------|------------------|
| busy       | Tight spin      | Continuous spin | None             |
| spin+back. | Spin+PAUSE      | Continuous spin | None             |
| adaptive   | Spin-then-futex | Hybrid          | Futex            |
| futex      | FUTEX_WAIT      | Sleeping        | Futex            |
| eventfd    | poll+read       | Sleeping        | poll/eventfd     |
| io_uring   | submit/wait     | Sleeping        | poll/io_uring    |

d) 4. *futex.*: The consumer stores 1 to `consumer_sleeping` and calls `SYS_futex(FUTEX_WAIT)`. The kernel checks the address atomically; if it still reads 1, the thread is suspended. The producer stores 0 and calls `FUTEX_WAKE`. One system call per sleep and per wakeup event.

e) 5. *eventfd.*: The consumer polls an eventfd file descriptor and then reads to consume the wakeup token. The producer writes 1 to the eventfd. The eventfd is compatible with `epoll/select`, easing integration into existing event-loop frameworks.

f) 6. *io\_uring.*: The consumer submits an `IORING_OP_READ SQE` targeting a named FIFO (`/tmp/uring_sig_fifo`) and blocks in `io_uring_submit_and_wait(ring, 1)`. The producer submits an `IORING_OP_WRITE SQE` to the same FIFO and calls `io_uring_submit`. The `io_uring` context can simultaneously monitor additional FDs, making it advantageous for frameworks managing many concurrent channels. Our primary ablation evaluates standard interrupt mode (`flags=0`); additionally, the implementation provides runtime support for `IORING_SETUP_SQPOLL` (SQPOLL mode via `USE_SQPOLL=1`), which offloads ring polling to an asynchronous kernel thread when the host permits SQPOLL operation.

## IV. EXPERIMENTAL METHODOLOGY

### A. Two-Suite Benchmark Framework

**Suite 1 — Wakeup-Mechanism Ablation.** The producer operates under three regime families: *saturated* (back-to-back sends), *bursty* (64-message bursts separated by 1 ms), and offered-load sweeps at 25, 50, 75, and 90 % of saturation. Payload sizes sweep across {64 B, 256 B, 1 KiB, 4 KiB, 16 KiB, 64 KiB, 256 KiB, 1 MiB}.  $N = 15$  measured runs per configuration (one warmup discarded). Suite 1 timestamps are collected on the pinned producer and consumer with `std::chrono::high_resolution_clock`.

**Suite 2 — Corrected Depth-1 Ping-Pong.** Queue depth is strictly enforced to 1: the initiator sends one message and blocks waiting for the echo server to reflect it before sending the next. Unlike Suite 1, both latency endpoints are timestamped on one core. Both  $t_{\text{start}}$  and  $t_{\text{end}}$  are recorded on **Initiator Core 1** using `clock_gettime(CLOCK_MONOTONIC_RAW)`. Single-trip latency:

$$L_{\text{trip}} = \frac{t_{\text{end}} - t_{\text{start}}}{2} \quad [\mu\text{s}]$$

The suite discards 10 000 warmup trips and uses size-scaled measured samples: 100 000 rounds through 1 KiB, 50 000 at

TABLE III  
REFERENCE TEST ENVIRONMENT

| Parameter       | Value                                                |
|-----------------|------------------------------------------------------|
| System          | ASUS Vivobook K3605ZF                                |
| OS / Kernel     | Ubuntu Linux / 6.17.0-35-generic                     |
| CPU             | Intel Core i5-12500H (Alder Lake, 12th Gen)          |
| Architecture    | Hybrid P/E-core                                      |
| Cores/Threads   | 12 Physical / 16 Logical                             |
| Rated Frequency | 400–4500 MHz; Turbo disabled during runs             |
| L1d Cache       | 448 KiB (12 instances)                               |
| L2 Cache        | 9 MiB (6 instances)                                  |
| L3 Cache        | 18 MiB (shared)                                      |
| RAM             | 16 GiB DDR4-3200                                     |
| Storage         | 512 GB Samsung NVMe SSD                              |
| Compiler        | g++ -O3, C++17 (ablation) / C++20 (ping-pong)        |
| liburing        | System package, Ubuntu 24.04                         |
| CPU Governor    | performance;<br><code>intel_pstate.no_turbo=1</code> |

4 KiB, 20 000 at 16 KiB, 10 000 at 64 KiB, 5 000 at 256 KiB, and 2 000 at 1 MiB. P50, P90, P99, P99.9, and a 95 % confidence interval are reported.

### B. Hardware and System Configuration

**Core affinity.** Producer/Initiator pinned to Core 1; Consumer/Echo-Server pinned to Core 2 via `sched_setaffinity`. Both cores share the 18 MiB L3 cache, keeping the ring buffer warm across the inter-process boundary.

**CPU tuning and frequency-pinned replication.** The final ablation and ping-pong runs were collected on the native Linux host with Core 1 and Core 2 pinned, the performance governor on both cores, and `intel_pstate.no_turbo=1`. These settings are recorded in `data/environment_ablation.txt` and `data/environment_pingpong.txt`. Core isolation through `isolcpus/nohz_full` was not used; residual scheduler and idle-state effects therefore remain a limitation.

### C. Statistical Methodology

Following established systems-performance measurement practice [3], we report central tendency, tail percentiles, and uncertainty rather than a single best run. For each (mechanism, payload size) pair we compute:

$$\bar{T} = \frac{1}{N} \sum_{k=1}^N T_k \quad [\text{GiB/s}], \quad \bar{L} = \frac{1}{M} \sum_{i=1}^M L_i \quad [\mu\text{s}]$$

$$\sigma_L = \sqrt{\frac{1}{M} \sum_{i=1}^M (L_i - \bar{L})^2}$$

where  $N = 15$  is the Suite 1 run count and  $M$  is the number of measured round trips in Suite 2. Ping-pong 95 % confidence intervals use  $\bar{L} \pm 1.96\sigma_L/\sqrt{M}$ , matching the published analysis code.

TABLE IV  
SUITE 1 DATA VOLUME PER RUN

| Configuration       | $\leq 1$ KiB | 4–64 KiB | $\geq 256$ KiB |
|---------------------|--------------|----------|----------------|
| Each wakeup variant | 16 MiB       | 128 MiB  | 512 MiB        |

TABLE V  
SINGLE-TRIP PING-PONG LATENCY DISTRIBUTIONS ( $\mu$ s). EACH ROW REPORTS P50, P90, P99, P99.9, AND MEAN  $\pm$  95% CI FROM THE CANONICAL ROOT SUMMARY CSVs.

| Transport / Variant                           | P50            | P90     | P99     | P99.9    | Mean $\pm$ CI       |
|-----------------------------------------------|----------------|---------|---------|----------|---------------------|
| <i>Message Size: 64 B — lower is better</i>   |                |         |         |          |                     |
| shm (adaptive)                                | <b>0.345</b>   | 0.439   | 0.505   | 1.189    | 0.363 $\pm$ 0.002   |
| shm (busy_poll)                               | 0.381          | 0.500   | 0.851   | 1.584    | 0.414 $\pm$ 0.003   |
| shm (spin_back)                               | 0.727          | 1.284   | 1.317   | 1.823    | 0.846 $\pm$ 0.002   |
| shm (futex)                                   | 5.001          | 5.097   | 6.395   | 8.872    | 5.033 $\pm$ 0.002   |
| shm (eventfd)                                 | 5.039          | 5.169   | 6.444   | 8.832    | 5.081 $\pm$ 0.002   |
| pipe                                          | 5.290          | 5.400   | 7.016   | 10.984   | 5.345 $\pm$ 0.005   |
| posix_mq                                      | 5.567          | 5.685   | 7.361   | 9.824    | 5.629 $\pm$ 0.002   |
| unix_socket                                   | 5.496          | 7.214   | 8.196   | 11.330   | 5.900 $\pm$ 0.017   |
| shm (io_uring)                                | 8.150          | 8.426   | 10.351  | 14.347   | 8.236 $\pm$ 0.003   |
| <i>Message Size: 4 KiB — lower is better</i>  |                |         |         |          |                     |
| shm (adaptive)                                | <b>2.263</b>   | 2.339   | 2.422   | 5.072    | 2.271 $\pm$ 0.001   |
| shm (busy_poll)                               | 2.306          | 2.413   | 3.576   | 8.527    | 2.353 $\pm$ 0.004   |
| shm (spin_back)                               | 2.669          | 2.740   | 2.947   | 5.474    | 2.662 $\pm$ 0.002   |
| shm (futex)                                   | 6.735          | 6.899   | 8.575   | 10.677   | 6.808 $\pm$ 0.003   |
| shm (eventfd)                                 | 6.809          | 7.137   | 8.403   | 10.874   | 6.898 $\pm$ 0.011   |
| pipe                                          | 7.147          | 7.470   | 8.825   | 12.966   | 7.248 $\pm$ 0.004   |
| posix_mq                                      | 8.746          | 9.073   | 10.751  | 13.212   | 8.824 $\pm$ 0.007   |
| shm (io_uring)                                | 9.855          | 10.119  | 12.056  | 15.679   | 9.923 $\pm$ 0.005   |
| unix_socket                                   | 11.864         | 12.657  | 13.609  | 19.349   | 11.093 $\pm$ 0.016  |
| <i>Message Size: 64 KiB — lower is better</i> |                |         |         |          |                     |
| shm (busy_poll)                               | <b>23.293</b>  | 23.845  | 26.982  | 48.473   | 23.280 $\pm$ 0.038  |
| shm (adaptive)                                | 23.888         | 24.006  | 25.829  | 27.300   | 23.858 $\pm$ 0.013  |
| shm (spin_back)                               | 24.203         | 24.293  | 25.939  | 27.925   | 24.155 $\pm$ 0.014  |
| shm (eventfd)                                 | 27.244         | 27.478  | 29.342  | 31.348   | 27.228 $\pm$ 0.014  |
| shm (futex)                                   | 27.227         | 27.464  | 29.242  | 31.836   | 27.216 $\pm$ 0.019  |
| unix_socket                                   | 28.284         | 29.270  | 32.880  | 37.986   | 28.362 $\pm$ 0.022  |
| shm (io_uring)                                | 29.409         | 29.689  | 31.802  | 33.743   | 29.262 $\pm$ 0.019  |
| pipe                                          | 35.861         | 36.380  | 40.106  | 41.696   | 35.935 $\pm$ 0.017  |
| posix_mq                                      | 35.869         | 36.721  | 39.651  | 42.714   | 35.988 $\pm$ 0.038  |
| <i>Message Size: 1 MiB — lower is better</i>  |                |         |         |          |                     |
| unix_socket                                   | <b>241.094</b> | 248.331 | 261.743 | 365.375  | 242.922 $\pm$ 0.291 |
| shm (busy_poll)                               | 267.243        | 276.180 | 285.295 | 295.643  | 269.615 $\pm$ 0.205 |
| shm (spin_back)                               | 267.366        | 276.029 | 285.168 | 297.980  | 269.596 $\pm$ 0.203 |
| shm (eventfd)                                 | 276.786        | 283.946 | 297.494 | 327.397  | 278.874 $\pm$ 0.316 |
| shm (futex)                                   | 278.034        | 286.596 | 296.704 | 335.171  | 280.502 $\pm$ 0.236 |
| shm (adaptive)                                | 278.976        | 288.360 | 300.819 | 314.115  | 282.830 $\pm$ 2.696 |
| shm (io_uring)                                | 281.154        | 289.455 | 301.202 | 335.805  | 283.185 $\pm$ 0.256 |
| pipe                                          | 473.293        | 482.965 | 542.826 | 1429.330 | 479.538 $\pm$ 3.410 |
| posix_mq                                      | N/A            | N/A     | N/A     | N/A      | N/A                 |

## D. Workload Volumes

### V. EMPIRICAL RESULTS

#### A. Unloaded Ping-Pong Latency (Suite 2)

Table V reports single-trip latency distributions (P50, P90, P99, P99.9, and mean  $\pm$  95% CI) at all four message sizes. Values are drawn consistently from the root data/pingpong\_\*\_summary.csv files; legacy duplicates and the conflicting aggregate file are not used.

##### Key observations:

(1) **14.5 $\times$  spinning advantage at small messages.** At 64 B, adaptive (0.345  $\mu$ s) and busy\_poll (0.381  $\mu$ s) are at least 13 $\times$  lower than any kernel-sleeping variant; adaptive is 14.5 $\times$  lower than futex. This gap includes the system call, scheduler wakeup, and context-switch path needed to resume a sleeping consumer.

TABLE VI  
STREAMING THROUGHPUT (GiB/s) — SATURATED REGIME,  $N = 15$  RUNS

| Wakeup Variant | 64 B         | 4 KiB        | 64 KiB       | 1 MiB       |
|----------------|--------------|--------------|--------------|-------------|
| busy_poll      | 0.316        | 10.17        | <b>15.75</b> | <b>8.65</b> |
| spin_backoff   | <b>0.344</b> | 10.16        | 14.54        | 8.61        |
| adaptive       | 0.338        | 10.43        | 15.53        | 8.36        |
| eventfd        | 0.262        | 9.54         | 13.74        | 8.37        |
| futex          | 0.300        | <b>10.68</b> | 15.60        | 8.18        |
| io_uring       | 0.332        | 9.92         | 13.37        | 8.12        |
| uring (SQPOLL) | 0.109        | 4.22         | 6.66         | 4.45        |

(2) **Kernel-sleeping variants remain in the same latency class.** futex (5.001  $\mu$ s), eventfd (5.039  $\mu$ s), and io\_uring (8.150  $\mu$ s) span 5.0–8.2  $\mu$ s P50 at 64 B. The higher io\_uring value is consistent with additional SQ/CQ ring management. These figures come from the canonical 100,000-trip distributions.

(3) **POSIX MQ is valid through 64 KiB in the recorded run.** The repository’s 64 KiB summary records 10,000 round trips (P50 35.869  $\mu$ s). The 256 KiB and 1 MiB rows are explicitly marked unsupported by the host POSIX-MQ message-size limit and are reported as N/A.

(4) **UNIX Socket wins at 1 MiB.** 241.09  $\mu$ s vs. 267.2–281.2  $\mu$ s for SHM variants. On this host, the socket’s buffering path outperforms the fixed-slot SHM configuration at this payload size; the experiment does not isolate a single kernel cause.

#### B. Ablation: Wakeup Variant Comparison (Suite 2)

At 1 MiB, all six SHM variants converge near 267–281  $\mu$ s (Table V). Copying 1 MiB across the ring dominates all wakeup mechanism overhead, making wakeup selection irrelevant at that message size. Wakeup mechanism matters only when message service time is short enough to be latency-dominated.

#### C. Streaming Throughput: Saturated Regime (Suite 1)

Table VI presents mean throughput; Figure 2 visualises the full size sweep.

##### Key observations:

(1) **Saturated throughput is primarily data-path bound.** Under fully saturated load the ring rarely empties, so the wakeup primitive is rarely invoked. The six variants remain in the same order of magnitude, with the refreshed 64 KiB results spanning 13.37–15.75 GiB/s.

(2) **The controlled io\_uring result is 13.37 GiB/s at 64 KiB.** This value, rather than the repository’s older 28.17 GiB/s direct-harness result, is used for the paper’s throughput conclusions. Section V-E explains why the two harnesses cannot be combined.

(3) **Large messages converge.** At 1 MiB the six refreshed variants span 8.12–8.65 GiB/s. Payload copying and the fixed 64-slot ring dominate the wakeup mechanism at this size.

(4) **Small-message io\_uring overhead at 64 B.** At 64 B payload, the refreshed variants span 0.262–0.344 GiB/s. This variation reflects fixed per-message coordination overhead. At

TABLE VII  
BURSTY-REGIME WAKEUP AND END-TO-END LATENCIES AT 64 B

| Variant           | E2E P50               | Wakeup P50                 | CPU $\mu\text{s}/\text{msg}$ |
|-------------------|-----------------------|----------------------------|------------------------------|
| busy_poll         | 0.260 $\mu\text{s}$   | <b>0.022</b> $\mu\text{s}$ | 17.254                       |
| spin_back.        | 0.246 $\mu\text{s}$   | 0.094 $\mu\text{s}$        | 17.236                       |
| adaptive          | 126.008 $\mu\text{s}$ | 1029.26 $\mu\text{s}$      | 1.431                        |
| futex             | 126.357 $\mu\text{s}$ | 1091.92 $\mu\text{s}$      | 0.546                        |
| eventfd           | 126.277 $\mu\text{s}$ | 1093.81 $\mu\text{s}$      | 0.570                        |
| io_uring          | 126.658 $\mu\text{s}$ | 1094.98 $\mu\text{s}$      | 0.680                        |
| uring<br>(SQPOLL) | 5.130 $\mu\text{s}$   | 1058.47 $\mu\text{s}$      | 0.218                        |

4 KiB and above, payload copying dominates the observed throughput range.

**(5) SQPOLL is reported separately.** The single-channel SQPOLL configuration is slower throughout this saturated sweep (0.109–6.66 GiB/s over the reported sizes). This configuration-level result does not establish a general SQPOLL throughput limit; batching and poll-thread affinity remain outside the experiment.

#### D. Bursty Regime: Pure Wakeup Latency Ablation (Suite 1)

In the bursty regime, the consumer empties the ring and enters a sleep state before each burst. Table VII isolates wakeup latency per variant.

**Frequency-controlled interpretation.** The final bursty and offered-load runs were collected with the performance governor and Turbo disabled. This removes dynamic P-state frequency scaling as an explanation for the approximately millisecond-scale low-load values, although C-state exit and scheduler delay remain part of the measured end-to-end wakeup path. The close values for futex, eventfd, and interrupt-mode io\_uring support the limited conclusion that no configuration has a decisive advantage for a single SPSC sleeping wakeup on this platform.

**SQPOLL check.** SQPOLL was run as a separately labelled configuration. At 64 B its bursty wakeup P50 was 1058.5  $\mu\text{s}$ , versus 1095.0  $\mu\text{s}$  for interrupt-mode io\_uring; at 90% offered load the values were 106.9 and 103.7  $\mu\text{s}$ , respectively. SQPOLL does not provide a material wakeup-latency improvement in this single-channel workload. Its complete 64 B comparison is included in Tables VII and VIII. The apparently discordant 5.130  $\mu\text{s}$  SQPOLL E2E P50 in Table VII is not a 25 $\times$  wakeup result: wakeup P50 times the empty-ring wait beginning before the next burst and therefore includes the intentional 1 ms inter-burst gap, whereas E2E P50 starts at each message’s slot.send\_ns publication timestamp and is calculated over all messages. The low SQPOLL E2E median consequently shows that most SQPOLL messages incurred little within-burst queue residence; because this suite does not trace SQ-thread service order or per-burst scheduler alignment, we do not attribute the disparity to an intrinsic SQPOLL wakeup advantage.

As shown in Table VIII, under offered-load sweeps (25%, 50%, 75%, 90% of saturated capacity), spinning mecha-

TABLE VIII  
OFFERED-LOAD SWEEP: MEDIAN WAKEUP LATENCY ( $\mu\text{s}$ ) AT 64 B.

| Variant        | 25%         | 50%         | 75%         | 90%         | Sat.        |
|----------------|-------------|-------------|-------------|-------------|-------------|
| busy_poll      | <b>0.02</b> | <b>0.02</b> | <b>0.02</b> | <b>0.02</b> | <b>0.02</b> |
| spin_backoff   | 0.09        | 0.09        | 0.09        | 0.09        | 0.09        |
| adaptive       | 1374.2      | 619.0       | 195.3       | 46.7        | 0.09        |
| futex          | 1433.7      | 669.9       | 254.3       | 103.2       | 0.26        |
| eventfd        | 1433.0      | 670.1       | 253.1       | 103.3       | 0.28        |
| io_uring       | 1441.0      | 678.3       | 254.7       | 103.7       | 0.93        |
| uring (SQPOLL) | 1258.4      | 658.3       | 257.6       | 106.9       | 3.97        |

nisms (busypoll and spinbackoff) maintain fixed sub-microsecond latencies regardless of offered load. For kernel-sleeping variants (futex, eventfd, iouring), median latency dynamically scales down as load increases—dropping from  $\approx 1374$ – $1441 \mu\text{s}$  at 25% load down to  $\approx 103 \mu\text{s}$  at 90% load as the shorter inter-arrival gaps reduce opportunities for deep idle and scheduler descheduling. At saturation, io\_uring’s higher latency (0.93  $\mu\text{s}$ ) compared to futex/eventfd (0.26–0.28  $\mu\text{s}$ ) reflects the constant overhead of SQE/CQE ring management on the non-blocking fast path. SQPOLL follows the same load-dependent trend (1258.4 to 106.9  $\mu\text{s}$  from 25% to 90%) but records 3.97  $\mu\text{s}$  at saturation, so it is not a zero-overhead replacement in this configuration. adaptive hybrid spinning captures the best of both worlds, reducing low-load sleep penalties while achieving pure spinning performance (0.09  $\mu\text{s}$ ) under saturation.

The bursty-regime protocol (Table VII: 64-message burst, then a fixed 1 ms idle gap) and the offered-load sweep (Table VIII: periodic gaps calibrated to a fixed fraction of saturated rate) are deliberately different idle-gap structures and are not directly comparable message-for-message. The fixed 1 ms gap in the bursty protocol is shorter than the mean inter-arrival gap implied by 25% of saturated throughput at 64 B, so the CPU has less time to fall to a deeper idle C-state before the next burst arrives—producing the lower  $\approx 1090$ – $1104 \mu\text{s}$  wakeup latency in Table VII relative to Table VIII’s 25%-load row. As offered load rises toward saturation in Table VIII, the mean inter-arrival gap shrinks below the bursty protocol’s fixed 1 ms gap, which is why the 90%-load row ( $\approx 103 \mu\text{s}$ ) falls below the bursty-regime figures despite both nominally representing “low load” conditions in isolation. The two tables should be read as separate experimental protocols probing the same underlying C-state ramp-up effect from different angles, not as repeated measurements of the same condition.

#### E. Controlled Throughput Provenance

The repository retains two throughput-data families. The legacy direct transport harness uses a fixed 2 GiB transfer per size and separate transport source trees; its io\_uring\_results.csv records 28.17 GiB/s at 64 KiB. The controlled ablation uses the bit-identical ring, size-scaled 16/128/512 MiB runs, and additional wakeup telemetry; it records 13.37 GiB/s for interrupt-mode io\_uring at the same size. The legacy CSV predates the fixed-frequency environment records, so the observed 2.1 $\times$  difference cannot

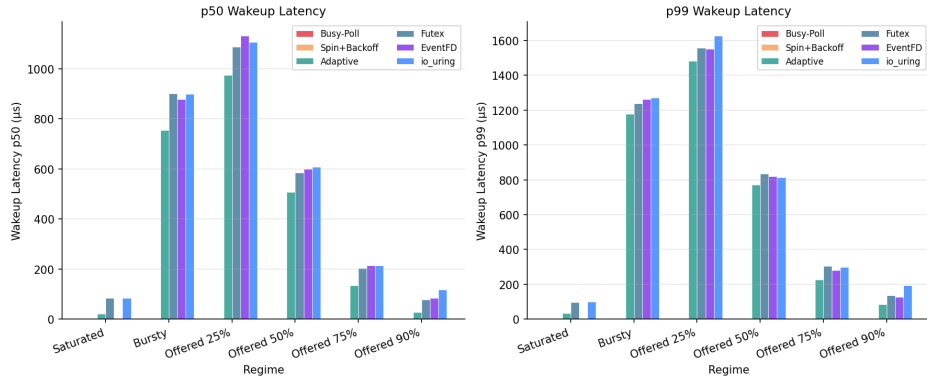


Fig. 1. Bursty-Regime Wakeup Latency Across Six SHM Variants. Spinning variants maintain near-zero wakeup latency while continuously spinning. Kernel-sleeping variants (`futex`, `eventfd`, `io_uring`) remain close at 1.092–1.095 ms in the frequency-pinned replication.

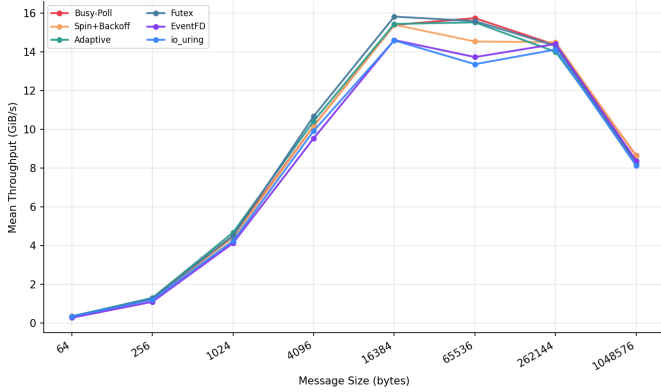


Fig. 2. Controlled saturated-throughput sweep for the six bit-identical SHM-ring wakeup variants ( $N = 15$ ). This figure visualises the same data family as Table VI; SQPOLL is reported separately in that table.

TABLE IX  
HARDWARE CACHE-MISS RATES FROM THE RECORDED PERF RUNS

| Mechanism | L1 Miss % | LLC Miss % | Relative to Pipe |
|-----------|-----------|------------|------------------|
| Pipe      | 2.42 %    | 33.6 %     | 1.00×            |
| Socket    | 1.12 %    | 31.5 %     | 0.94×            |
| MQ        | N/A       | 3.2 %      | 0.10×            |
| SHM Ring  | 3.97 %    | 4.97 %     | 0.15×            |

be assigned to a wakeup mechanism or to one harness feature. Consequently, the final throughput conclusions and Figure 2 use only the refreshed controlled ablation. The legacy direct CSVs remain in the artifact for provenance but are not used for cross-harness performance claims. Kernel-mediated transports are compared against the ring using the controlled depth-1 latency suite in Table V.

#### F. Cache and Memory Hierarchy Analysis

The 4.97 % LLC miss rate is consistent with the SHM ring retaining a cache-friendly active working set. Pipe and socket record 30–34 % LLC miss rates in the corresponding perf runs. The `alignas(64)` layout also prevents head and tail from sharing a cache line.

## VI. DISCUSSION

### A. What `io_uring` Actually Contributes to IPC

For a single SPSC ring channel, the completed depth-1 experiment shows that interrupt-mode `io_uring` does not reduce the measured per-event cost relative to the simpler blocking mechanisms. At 64 B it produces a  $8.15 \mu\text{s}$  P50, versus `futex`’s  $5.00 \mu\text{s}$ , consistent with SQE/CQE submission overhead. The bursty observations place all three kernel-sleeping variants in a similar range under recorded fixed-frequency settings. The SQPOLL result is also close, so this workload exposes no decisive `io_uring`-mode advantage over `futex` or `eventfd`.

The primary performance advantage of SHM-based IPC originates entirely from replacing kernel-copy data paths with shared-memory `memcpy`—not from the wakeup primitive. Performance breaks down into a two-layer stack:

- **Data path** (dominant): shared memory vs. kernel-copy. Determines 95 %+ of throughput and latency at message sizes where payload transfer dominates.
- **Wakeup layer** (secondary): spinning vs. blocking. Dominates only when the ring is genuinely empty between bursts.

`io_uring`’s genuine advantages over `futex`/`eventfd` for this use case are: (1) a unified multi-FD event loop replacing  $O(N)$  `futex_wait` calls with a single `io_uring_submit_and_wait`; (2) batched submission amortizing kernel-entry cost across multiple wakeup signals; and (3) a clear SQPOLL upgrade path that can remove the submission syscall via `IORING_SETUP_SQPOLL`.

### B. Architectural Guidance by Use Case

a) *High-Frequency Trading / Sub- $\mu\text{s}$  Latency.*: Use SHM ring with `busy_poll` or `adaptive`. Both achieve  $P50 \approx 345 \text{ ns}$  and  $P99 \leq 1.2 \mu\text{s}$  at 64 B. Busy polling requires a dedicated spinning core; `adaptive` waiting blocks after its spin window. `adaptive` is preferred for intermittent workloads: it spins for up to 500 iterations and falls back to `futex` only when the ring is genuinely empty, reducing measured CPU time during burst-idle operation while preserving the hot path.

b) *Cloud Microservices / Energy-Constrained Environments.*: Use SHM ring with `futex` or `eventfd`. Both use blocking waits and deliver approximately  $5\ \mu\text{s}$  P50 and sub- $6.5\ \mu\text{s}$  P99 at 64 B in depth-1 ping-pong. `eventfd` integrates naturally into existing `poll/epoll` event loops without `futex`-specific code, making it the lower-friction choice for retrofit into existing stacks.

c) *Async Event-Driven Frameworks.*: Use SHM ring with `io_uring`. Wakeup latency matches `futex/eventfd` within the same kernel-sleeping class ( $5.0\text{--}8.2\ \mu\text{s}$  P50), but the `io_uring` context unifies IPC ring monitoring with network I/O, disk I/O, and timer events in a single `submit_and_wait` call. This eliminates the complexity of maintaining parallel `epoll` and `futex` contexts and provides a clear upgrade path to SQPOLL mode.

d) *Large Payloads (>1 MiB) or Deep Buffering.*: Use UNIX Domain Sockets. At 1 MiB, UNIX Socket achieves  $241.1\ \mu\text{s}$  P50 vs.  $267.2\text{--}281.2\ \mu\text{s}$  for SHM ring variants. The measurements establish the configuration-level result but do not isolate a single kernel mechanism.

### C. Ring-Depth Constraint at 1 MiB

At 1 MiB, the refreshed saturated ablation variants span  $8.12\text{--}8.65\ \text{GiB/s}$ . This convergence is consistent with the fixed 64-slot ring geometry, although the current experiment does not include a controlled ring-depth sweep. With each slot sized at 1 MiB, the ring holds at most 64 MiB of in-flight data. The producer stalls as soon as all 64 slots are occupied while the consumer processes each 1 MiB slot sequentially.

The fixed  $N = 64$  slot count therefore bounds in-flight buffering ( $64 \times 1\ \text{MiB} = 64\ \text{MiB}$ ). The present measurements do not separate that capacity effect from copy bandwidth, checksum cost, or socket buffering, so the ring-depth explanation remains a hypothesis rather than a demonstrated causal result.

**No refreshed interrupt-mode outlier.** In the refreshed saturated ablation, interrupt-mode `io_uring` reaches  $8.12\ \text{GiB/s}$  at 1 MiB, within the  $8.12\text{--}8.65\ \text{GiB/s}$  range of all six variants. The earlier  $4.66\ \text{GiB/s}$  value is not used for the final conclusions.

Increasing `NUM_SLOTS` to 256 would raise in-flight capacity to 256 MiB. A controlled depth sweep is required to determine whether that change recovers throughput at 1 MiB; this is left as future work.

### D. Comparison with Related Measurement Studies

Our findings agree with Didona et al. [7] that `io_uring` provides no inherent advantage for unbatched single-operation IPC when compared to simpler synchronous primitives. Similarly, in our depth-1 IPC ping-pong benchmark (Suite 2), per-message submission overhead makes `io_uring` slightly slower than `futex`, matching its latency only when wakeups are amortized across bursty arrivals (Suite 1). In contrast to Ren and Trivedi [8], who attribute speedups to `io_uring`'s reduced per-operation overhead for NVMe I/O, our experiment shows this benefit does not transfer to shared-memory IPC wakeup: the NVMe path benefit arises from batching DMA

descriptor submissions, whereas SPSC wakeup is inherently one-at-a-time (one signal per empty-ring event). Batching is only applicable when multiple channels empty simultaneously, which requires a multi-ring architecture outside the scope of this study.

Practitioner implementations such as Aeron [5] use a dedicated “media driver” thread that busy-spins on all channels, effectively implementing `busy_poll` at the framework level. Our data support this design choice: `busy_poll` achieves  $0.022\ \mu\text{s}$  wakeup latency vs.  $1092\text{--}1095\ \mu\text{s}$  for the interrupt-mode sleeping variants. The trade-off is one dedicated core per media driver; for applications that can afford this, it provides the lowest measured wakeup latency on this platform.

## VII. THREATS TO VALIDITY

- 1) **Single node and microarchitecture.** All experiments ran on one Intel 12th-Gen x86\_64 laptop. Results on ARM64, AMD Zen, or multi-socket NUMA may differ due to different cache-coherency protocols and scheduler implementations.
- 2) **SPSC topology only.** Results apply to Single-Producer Single-Consumer queues. MPMC rings require CAS loops introducing contention absent in SPSC.
- 3) **Synthetic payload workload.** Payloads consist of repeated-byte patterns. Real application payloads (protobuf, encrypted frames) may shift the bottleneck to serialisation rather than ring mechanics.
- 4) **Residual idle-state and scheduler effects.** The final runs record performance and `no_turbo=1`, but do not use `isolcpus/nohz_full`. Thus, the result is specific to this native Linux laptop and should not be generalized to all CPUs.
- 5) **Suite 1 clock placement.** Suite 1 latency fields use cross-core `high_resolution_clock` timestamps. The corrected depth-1 Suite 2 avoids this limitation by placing both `CLOCK_MONOTONIC_RAW` timestamps on the initiator core.
- 6) **POSIX MQ coverage.** The recorded configuration includes a valid 64 KiB run. The 256 KiB and 1 MiB rows are marked unsupported because of the host message-size limit.
- 7) **SQPOLL scope.** SQPOLL is evaluated as a separate 64-slot, single-channel configuration. Its result does not generalize to multi-FD batching or other SQPOLL thread-affinity choices.
- 8) **Tail-percentile provenance.** Table V is derived only from the root `data/pingpong*_summary.csv` files. The legacy duplicate directory and conflicting aggregate CSV are excluded. POSIX MQ has no valid 1 MiB result.

## VIII. CONCLUSION & FUTURE WORK

This paper presented a rigorous, controlled measurement study of wakeup mechanism performance in lock-free shared-memory SPSC IPC. By holding the ring buffer design constant

and varying only the sleep/wakeup primitive, we isolated the true contribution of `io_uring` to IPC latency.

#### Summary of findings:

- 1) **The shared-memory data path drives the performance gains.** Spinning on a cache-aligned ring delivers 345 ns P50 at 64 B. In the controlled saturated ablation, the six primary variants reach 13.37–15.75 GiB/s at 64 KiB.
- 2) **Interrupt-mode wakeup result is deliberately scoped.** In strict depth-1 ping-pong, interrupt-mode `io_uring` pays a modest per-event overhead relative to `futex`. The frequency-pinned bursty and offered-load replication shows closely grouped sleeping variants, while `SQPOLL` also does not materially improve the 64 B wakeup result in this single-channel design.
- 3) **Spinning variants dominate sub- $\mu$ s latency requirements.** `adaptive` is the preferred choice: it achieves 345 ns hot-path latency and gracefully reduces CPU consumption during idle periods.
- 4) **The 1 MiB throughput convergence may reflect ring depth.** A controlled ring-depth sweep is needed to test whether increasing `NUM_SLOTS` recovers the throughput advantage.
- 5) **Cache hierarchy alignment is critical.** The SHM ring achieves a 4.97 % LLC miss rate vs. 30–34 % for kernel-copy mechanisms, consistent with reduced cache pressure in the shared-memory data path.

#### Future directions:

- *Multi-FD SQPOLL mode.* Evaluate whether batching several channels or changing the `SQPOLL` thread affinity changes the single-channel result reported here.
- *MPMC extension.* A lock-free MPMC ring with sequence-lock or CAS-based reservation extends the ablation to multi-producer settings.
- *Cross-NUMA evaluation.* Profiling with producer on NUMA node 0 and consumer on NUMA node 1 would quantify LLC miss penalties and motivate NUMA-aware ring partitioning.
- *RDMA baseline.* An InfiniBand one-sided write baseline contextualises these results within HPC cluster IPC.
- *Production traffic models.* Replacing synthetic payloads with real-world Kafka message logs or gRPC frame traces would validate whether serialisation overhead shifts the bottleneck off ring mechanics.

#### REFERENCES

- [1] J. Axboe, “Efficient I/O with `io_uring`,” Linux Kernel Documentation, Tech. Rep., 2019. Available: [https://kernel.dk/io\\_uring.pdf](https://kernel.dk/io_uring.pdf)
- [2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Addison-Wesley Professional, 2013.
- [3] B. Gregg, *Systems Performance: Enterprise and the Cloud*, 2nd ed. Addison-Wesley, 2020.
- [4] L. Lamport, “Specifying concurrent program modules,” *ACM Trans. Programming Languages and Systems*, vol. 5, no. 2, pp. 190–222, Apr. 1983.
- [5] M. Thompson, D. Farley, M. Barker, P. Gee, and A. Stewart, “Disruptor: High performance alternative to bounded queues for exchanging data between concurrent threads,” LMAX Exchange, White Paper, 2011.
- [6] DPDK Project, “DPDK Programmer’s Guide: Ring Library,” 2024. Available: [https://doc.dpdk.org/guides/prog\\_guide/ring\\_lib.html](https://doc.dpdk.org/guides/prog_guide/ring_lib.html)
- [7] D. Didona, J. Pfefferle, N. Ioannou, B. Metzler, and A. Trivedi, “Understanding modern storage APIs: A systematic study of `libaio`, `SPDK`, and `io_uring`,” in *Proc. 15th ACM International Systems and Storage Conference (SYSTOR ’22)*, Haifa, Israel, Jun. 2022, pp. 1–16.
- [8] Z. Ren and A. Trivedi, “Performance Characterization of Modern Storage Stacks: `POSIX I/O`, `libaio`, `SPDK`, and `io_uring`,” in *Proc. 3rd Workshop on Challenges and Opportunities of Efficient and Performant Storage Systems (CHEOPS ’23)*, Rome, Italy, May 2023.
- [9] M. Herlihy and N. Shavit, *The Art of Multiprocessor Programming*. Morgan Kaufmann, 2008.
- [10] Linux man-pages project, `futex(2)`, `eventfd(2)`, `io_uring(7)`, `mq_overview(7)`, `pipe(7)`, `unix(7)`, kernel.org, 2024. Available: <https://man7.org/linux/man-pages/>

**Reproducibility.** All source code, benchmark scripts, raw CSV data, and figure-generation scripts are publicly available at [https://github.com/Rahul09123/io\\_uring-IPC](https://github.com/Rahul09123/io_uring-IPC).